D. Race Condition & Critical Section (OSNo06 ต้น ๆ)

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: Race Condition = ปัญหาที่ 2 process แก้ข้อมูลแชร์พร้อมกันจนผลลัพธ์เพี้ยนตามใครแก้ก่อน-หลัง · Critical Section (ช่วงวิกฤต) = ช่วงโค้ดที่แตะข้อมูลแชร์ ห้ามใครเข้าพร้อมกัน

ความสำคัญกับข้อสอบ: ไล่ timeline การแย่งตัวแปร count (S0–S5), ท่องเงื่อนไข 3 ข้อ (Mutual Exclusion (เข้าถึงได้คนเดียว) / Progress / Bounded Waiting (รอได้จำกัดจำนวนครั้ง)), อธิบาย Peterson's Solution (โซลูชันสำหรับ 2 กระบวนการ) ว่าครบ 3 ข้อยังไง — ออกทั้งถูก/ผิดและไล่ค่า

ใช้จริงในคอมพิวเตอร์: ระบบธนาคารที่โอนเงินพร้อมกัน 2 คน, ตั๋วหนังที่นั่งสุดท้ายถูกซื้อพร้อมกัน, ตัวนับยอดวิว — ถ้าไม่ล็อก ข้อมูลเสียแบบใน timeline เลย

D1. จุดเริ่ม: Bounded Buffer (บัฟเฟอร์ขนาดจำกัด) ยังไม่มี sync (Producer (ฝั่งผู้ผลิตข้อมูล)-Consumer (ฝั่งผู้บริโภคข้อมูล))

Producer-Consumer แชร์ buffer วงแหวนขนาด BUFFER_SIZE ช่อง + ตัวนับ `count` — producer เขียนแล้ว count++ / consumer อ่านแล้ว count−−

```
/* Producer (สไลด์ p.3)                    Consumer (สไลด์ p.4) */
while (true) {                          while (true) {
    while (count == BUFFER_SIZE);           while (count == 0);   // busy wait
    buffer[in] = nextProduced;              nextConsumed = buffer[out];
    in = (in + 1) % BUFFER_SIZE;            out = (out + 1) % BUFFER_SIZE;
    count++;                                count--;
}                                       }
/* ปัญหา: ข้อมูลหาย (producer เขียนทับที่ยังไม่อ่าน) / อ่านซ้ำ — และ count ไม่ atomic */
```

D2. Race Condition — ไล่ S0–S5 ให้เห็นตา 🔥

`count++` ในระดับเครื่องแตกเป็น 3 คำสั่ง (ไม่ atomic!):

```
/* count++: */              /* count--: */
register1 = count;          register2 = count;
register1 = register1 + 1;  register2 = register2 - 1;
count = register1;          count = register2;
```

ไทม์ไลน์จากสไลด์ p.5 (เริ่ม count = 5):

จังหวะ	ใครทำอะไร	ค่าที่เกี่ยวข้อง
S0	producer: register1 = count	r1 = 5
S1	producer: r1 = r1 + 1	r1 = 6
S2	consumer: register2 = count	r2 = **5** ⚠️ (ยังเก่า!)
S3	consumer: r2 = r2 − 1	r2 = 4
S4	producer: count = r1	count = 6
S5	consumer: count = r2	count = **4** ❌ (ควรได้ 5)

💳 อุปมา

เงินในบัญชี 5,000 · คุณโอนเข้า +1,000 แฟนถอน −1,000 พร้อมกัน แอปให้ทั้งคู่เห็นยอด 5,000 พร้อมกันแล้วบันทึกทับ → เหลือ 4,000 เงินหายในอากาศ!

> 📖 **นิยามที่ต้องพิมพ์ได้**
> **Race condition**: หลาย process เข้าถึง/จัดการข้อมูลร่วมกันพร้อมกัน แล้ว**ผลลัพธ์ขึ้นกับลำดับการเข้าถึง** · โค้ดช่วงที่แตะข้อมูลแชร์ = **critical section (critical region)**

D3. Critical Section — เงื่อนไข 3 ข้อที่โซลูชันต้องผ่าน 🔥

เงื่อนไข	ความหมาย (พิมพ์ได้ต้องได้)
1. Mutual Exclusion	ถ้า P_i อยู่ใน CS (ช่วงวิกฤต) → **ห้ามมี process อื่นอยู่ใน CS ของมันพร้อมกัน**
2. Progress	ถ้า*ไม่มีใคร*อยู่ใน CS และมีคนอยากเข้า → ต้องเลือกจาก*คนที่อยากเข้าเท่านั้น* และตัดสินไม่ล่าช้าไม่สิ้นสุด (กันคนไม่สนมาขวาง / กันไม่มีใครเข้าเลย)
3. Bounded Waiting	มีขอบเขตว่า process หนึ่งถูก "ตัดหน้า" ได้**จำกัดกี่ครั้ง**หลังยื่นขอ (กัน starvation ของคิว CS)

โครงโค้ด: ENTRY SECTION → CRITICAL SECTION → EXIT SECTION → REMAINDER SECTION (วน while(true)) · สมมติ: แต่ละ process รันด้วยความเร็วไม่เป็นศูนย์ แต่**ไม่มีข้อสมมติเรื่องความเร็วสัมพัทธ์**

D4. Peterson's Solution (2 process)

แชร์ 2 ตัวแปร: `int turn` (ตาใคร) + `boolean flag[2]` (ใครอยากเข้า) — สมมติ LOAD/STORE **atomic**

```
/* โค้ดของ Pᵢ (อีกฝั่งคือ Pⱼ) — สไลด์ p.9–10 */
while (true) {
    flag[i] = TRUE;              // 1. ประกาศว่า "ฉันอยากเข้า!"
    turn = j;                    // 2. ยกสิทธิ์ให้เพื่อนก่อน (ความสุภาพ = หัวใจ)
    while (flag[j] && turn == j); // 3. รอเฉพาะเมื่อ: เพื่อนอยากเข้า "และ" ถึงตาเพื่อน

    CRITICAL SECTION

    flag[i] = FALSE;             // 4. ออกแล้ว — ปล่อยให้คนอื่นเข้าได้
    REMAINDER SECTION
}
```

เงื่อนไข	Peterson ทำไม่ผ่านทำไมยังไง
Mutual Exclusion ✓	ทั้งคู่ตั้ง turn พร้อมกันไม่ได้ — คนที่ตั้ง turn เป็นฝ่ายอื่น**เป็นคนสุดท้าย** ต้องรอก่อน
Progress ✓	เงื่อนไขรอต้องเจอ flag[j] = true จริง — คนที่ไม่สนใจเข้าไม่ขวาง
Bounded Waiting ✓	หลังออกจาก CS อีกฝั่งได้เข้าภายใน 1 รอบ (turn สลับเสมอ)

> 📌 **ในโลกจริง**
> Peterson เวิร์คในทฤษฎี แต่ CPU สมัยใหม่ (compiler optimization, out-of-order) อาจพัง — ใช้จริงด้วยฮาร์ดแวร์/ไลบรารี แต่**ออกสอบเยอะ**เพราะสอนหลักคิด

D5. Bakery Algorithm (n process) — จ่ายคิวร้านเบเกอรี่

- ก่อนเข้า CS จ่ายเลขคิว = max(number[0..n−1]) + 1 → คนถือเลข**น้อยสุด**เข้าก่อน · เลขซ้ำเทียบ **PID** มาก่อนชนะ
- เทียบแบบ lexicographical: `(a,b) < (c,d)` ถ้า a < c หรือ (a = c และ b < d) — (เลขคิว, pid)
- แชร์: `boolean choosing[n]`, `int number[n]` (เริ่ม false/0)

```
do {
    choosing[i] = true;                          // กำลังจ่ายเลข (ยังไม่นับคิวจริง)
    number[i] = max(number[0..n-1]) + 1;          // จ่ายเลขใหญ่กว่าใคร ๆ +1 (อาจซ้ำ)
    choosing[i] = false;
    for (j = 0; j < n; j++) {
        while (choosing[j]);                     // รอคนที่กำลังจ่ายเลขจบก่อน
        while ((number[j] != 0) && ((number[j],j) < (number[i],i)));
        // รอถ้า: เพื่อนถือคิวอยู่ และ (เลขน้อยกว่า หรือ เลขเท่าแต่ pid น้อยกว่า)
    }
    CRITICAL SECTION
    number[i] = 0;                               // ออก → ยกเลิกคิวตัวเอง
    REMAINDER SECTION
} while (true);
```

เลขคิวอาจกระโดด (1,2,3,3,3,4) — ไม่เป็นไร lexicographical จัดการเคสซ้ำ · รับประกันครบ 3 เงื่อนไขสำหรับ n process